

Analysis: GBus count

[View problem and solution walkthrough video](#)

Given the serving range of every GBus, we need to determine the number of GBuses serving a particular list of cities. If we start from the leftmost city and move towards the right, we can make the following observations.

1. There will be a certain point where the GBus starts to contribute to the total number of GBuses for a given city. This certain point is the left end of the GBus serving range (which is A_i for i -th GBus).
2. And there will be another certain point where the GBus is out of the serving range, and it will never come back again. This certain point is one city to the right of the right end of the GBus serving range (which is B_{i+1} for i -th GBus).

Based on the two observations above, let us start formulating a solution.

- For each city, we maintain the total number of GBuses whose left end point is at this city. Denote this value as X_i for i -th city.
- Similarly, we maintain the total number of GBuses whose right end point is one city to the left of this city. Denote this value as Y_i for i -th city.
- Now we can simply scan the cities from left to right. For i -th city, we add X_i to the current total number of GBuses (initially 0), which denotes the number of GBuses that starts to contribute to the total number at this point. Then, we subtract Y_i from the current total number of GBuses, which denotes the number of GBuses that no longer contribute to the total number at this point.
- After that, for each city from the query, we just output the total number pre calculated for that city.

The time complexity for the solution will be linearly dependent on the maximum number of cities that a GBus can serve, which can be at most 5000 given the constraints of both test sets.

Analysis: Record Breaker

[View problem and solution walkthrough video](#)

To check whether i is a record breaking day, we make two checks:

- The number of visitors on day i (V_i) must be greater than the number of visitors for any previous days.
- Either it is the last day OR the number of visitors on the day i are more than the day $i + 1$.

The second one is easy to check in constant time. Checking the first one is the difficult part of this problem. Here's how we would do that:

Test Set 1

For each element j such that ($1 \leq j < i$), check that the number of visitors on day j are less than number of visitors on day i . In other words ($V_j < V_i$). Hence, for each day we would compare it with all the previous days and it would take $O(N)$ time. Therefore, for N days, the time complexity of this solution would be $O(N^2)$.

Test Set 2

However, a solution that takes $O(N^2)$ time is not fast enough for Test Set 2, so we need a faster approach. Instead of comparing the number of visitors of day i against *all* the previous days one by one, we can compare the number of visitors of day i against the *greatest number of visitors* from all previous days. That reduces our processing time for each day from $O(N)$ to $O(1)$. Therefore, for N days, the time complexity of this solution would be $O(N)$, which is sufficiently fast for both Test Set 1 and Test Set 2.

Sample Code (C++)

```
int countRecordBreakingDays(vector<int> visitors) {
    int recordBreaksCount = 0;
    int previousRecord = 0;
    for(int i = 0; i < checkpoints.size(); i++) {
        bool greaterThanPreviousDays = i == 0 || visitors[i] > previousRecord;
        bool greaterThanFollowingDay = i == checkpoints.size()-1 || visitors[i] > visitors[i+1];
        if(greaterThanPreviousDays && greaterThanFollowingDay) {
            recordBreaksCount++;
        }
        previousRecord = max(previousRecord, visitors[i]);
    }
    return recordBreaksCount;
}
```

Analysis: Sample Problem

[View problem and solution walkthrough video](#)

Solving this problem involves a number of steps. First, we must determine the total number of candies we have available to distribute. We can calculate this by summing together the number of candies in each bag, $total := \sum_{i=1}^N C_i$. Once we have the total number of candies, we must determine the number of candies that will remain after evenly distributing as many as possible among M kids. We can calculate this by dividing the total number of candies by the number of kids. M may not evenly divide $total$, so $(total \div M)$ will have an integer part (the amount each kid receives) and a remainder part (the amount left over). Because we only care about the remainder, we can use the [modulo operation](#) which does exactly this.

Example solution:

```
Solve(N, M, C) {
    total = 0
    for i in 1:N {
        total = total + C[i]
    }
    remainder = modulo(total, M)
    return remainder
}
```

Limits and complexity:

With the input limits for this problem, the default integer size for most programming languages will be sufficient to perform all arithmetic without risk of overflow. Specifically, the largest possible value is $total = (N_{\max} \times C_{i\max}) = 10^8$, which is easily stored in a 32-bit signed integer ($int_{\max} \approx 2.147 \times 10^9$). If the limits were larger, we could take advantage of the transitive and symmetric properties of modular arithmetic to calculate the remainder provided by each bag, and the running remainder after each addition:

```
Solve(N, M, C) {
    remainder = 0
    for i in 1:N {
        remainder = remainder + modulo(C[i], M)
        remainder = modulo(remainder, M)
    }
    return remainder
}
```

This reduces the largest intermediate value to $2 \times (M_{\max} - 1)$ with only a small amount of extra computation. When solving other problems in this competition, you are likely to encounter limits that will require this kind of optimization.

Computational complexity is also an important factor to consider when deciding how to approach a problem. Both approaches presented here have the same complexity, $O(\mathbf{N})$, which is sufficient to solve the problem within the time limit. Other problems in this competition can often be solved using a variety of approaches, but only some will be sufficiently fast to solve all test sets within the time limit. An algorithm with complexity $O(\mathbf{G}^3)$ may work for a small test set with $\mathbf{G}_{max} = 20$, but that same algorithm will be too slow for a large test set with $\mathbf{G}_{max} = 10^6$.

In general, time and memory limits for the problems in this competition, and the bounds for their test sets, are chosen so that algorithms pass or fail based on their overall time and memory complexity order. In other words, if a problem has a time limit of 60 seconds, it is unlikely that a fast algorithm would take 40 seconds while a slow one took 80 seconds. Instead, you will find that a fast algorithm might take 5 seconds while a slow one takes 5 years. So if you find your solution exceeding the time limit for a problem, take a moment to reevaluate your algorithm. There may be another approach that is many times faster.

Analysis: Sherlock and Watson Gym Secrets

[View problem and solution walkthrough video](#)

The problem translates to given $\mathbf{A}$, $\mathbf{B}$, $\mathbf{N}$, $\mathbf{K}$, find number of pairs (i, j) which satisfy the following conditions:

1. $i \neq j$
2. $1 \leq i \leq \mathbf{N}$ and $1 \leq j \leq \mathbf{N}$
3. $(i^{\mathbf{A}} + j^{\mathbf{B}}) \bmod \mathbf{K} = 0$

Test Set 1

We can brute force through all possible pairs (i, j) which satisfy the first two conditions and calculate if $(i^{\mathbf{A}} + j^{\mathbf{B}}) \bmod \mathbf{K} = 0$. As $1 \leq i \leq \mathbf{N}$ and $1 \leq j \leq \mathbf{N}$, we have $\mathbf{N}^2$ pairs and for each pair we need to compute $(i^{\mathbf{A}} + j^{\mathbf{B}}) \bmod \mathbf{K}$. If we compute this naively, we will require $O(\mathbf{A} + \mathbf{B})$ operations, but we can use [exponentiation by squaring](#) to compute this efficiently in $O(\log(\mathbf{A}) + \log(\mathbf{B}))$.

Since we need to compute this sum for each pair (i, j) , we get $O(\mathbf{N}^2(\log(\mathbf{A}) + \log(\mathbf{B})))$.

Test Set 2

As $\mathbf{N}$ is quite large for this test set, the previous approach would not work here, Let us try another approach. For $(i^{\mathbf{A}} + j^{\mathbf{B}}) \bmod \mathbf{K} = 0$, we know that $j^{\mathbf{B}} \bmod \mathbf{K} = -i^{\mathbf{A}} \bmod \mathbf{K}$.

For each possible i , let us try to find how many such j exists.

Let us create an array L , where $L[x]$ denotes the number of possible values of j such that $j^{\mathbf{B}} \bmod \mathbf{K} = x$. Now, we can iterate over all possible values of i i.e. $(1, 2, 3, \dots, \mathbf{N})$ and add $L[-i^{\mathbf{A}} \bmod \mathbf{K}]$ to the answer. One more thing which is left to handle here is the condition $i \neq j$, for which we can simply check if $(i^{\mathbf{A}} + i^{\mathbf{B}}) \bmod \mathbf{K} = 0$ and decrement 1 from the answer.

The complexity of this approach would be $O(\mathbf{N}(\log(\mathbf{A}) + \log(\mathbf{B})))$ which is better than the previous approach but still exceeds the time limit. But we can try to optimize more from here.

You can note that even though i can take $\mathbf{N}$ possible values, $i \bmod \mathbf{K}$ can only take $\mathbf{K}$ possible values. Can we take advantage of this ?

We defined $L[x] =$ number of j such that $j^{\mathbf{B}} \bmod \mathbf{K} = x$. A more optimized way to construct L can be: Consider a variable $q, 0 \leq q < \mathbf{K}$. (The number of values $j \bmod \mathbf{K}$ can take), we can iterate over all possible values of q i.e. $(1, 2, 3, \dots, \mathbf{K})$ and increment $L[q^{\mathbf{B}} \bmod \mathbf{K}]$ by number of j 's such that $j \bmod \mathbf{K} = q$. This reduces the time complexity of constructing L from $O(\mathbf{N} \log(\mathbf{B}))$ to $O(\mathbf{K} \log(\mathbf{B}))$.

Similarly, instead of iterating over all possible values of i , Consider a variable $p, 0 \leq p < \mathbf{K}$ (The number of values $i \bmod \mathbf{K}$ can take), Let $z =$ number of i 's such that $i \bmod \mathbf{K} = p$, and add $z \times L[-p^{\mathbf{A}} \bmod \mathbf{K}]$ to the answer.

We still need to handle the condition of $i \neq j$, for which we can simply check for each p if $(p^{\mathbf{A}} + p^{\mathbf{B}}) \bmod \mathbf{K} = 0$ and subtract z from the answer.

As we are iterating over p and q which take only $\mathbf{K}$ distinct values instead of $\mathbf{N}$ distinct values for i and j , the complexity of this approach reduces to $O(\mathbf{K}(\log(\mathbf{A}) + \log(\mathbf{B})))$.

A small mistake which can happen in exponentiation by squaring is $x^0 = 1$, which would be wrong in case of $\mathbf{K} = 1$, hence take $x^0 = 1 \bmod \mathbf{K}$.

Analysis: Wiggle Walk

[View problem and solution walkthrough video](#)

Test set 1

A naive approach to this problem is to simply simulate what has been mentioned in the problem statement. In other words, for each instruction, we will keep moving the robot in the specified direction until it reaches an unvisited square. This approach is going to have a time complexity of $O(N^2)$, which is good enough for test set 1.

Test set 2

For this test set, the naive approach will not be good enough. The problem with the naive approach is that it visits a lot of already visited squares. In cases such as when the input alternates continuously between E and W , each instruction will visit all the previously visited squares. We need a way to compute the destination square for each instruction faster to reduce the complexity.

Let us use intervals to represent all the contiguous segments of visited squares in the same row. For example, if in a row r the robot has already visited squares $(r, 3)$, $(r, 5)$, $(r, 6)$, and $(r, 7)$, the visited squares in this row can be represented with the intervals $(3, 3)$ and $(5, 7)$.

With that in mind, consider we have a [set](#) of intervals for each row and each column of the grid to represent all the squares that have been visited in that particular row or column. Let us call them interval-sets. Initially, the robot has only visited the starting squares (S_R, S_C) it started in, so all these sets are empty except for the set corresponding to row S_R , which has a single interval (S_C, S_C) , and the set corresponding to the column S_C , which has a single interval (S_R, S_R) .

Using this data structure, we can quickly find the destination square for the robot for each instruction. Let the robot be located at (r, c) . For each instruction, we will look at the interval-set corresponding to the row r (if the instruction is W or E) or column c (if the instruction is N or S). Then, we will find the interval in this set that contains the robot's current position (there is guaranteed be one, because the square the robot is currently in is already considered to be visited). Once we find that interval, we will immediately know the location of the next unvisited squares in either direction, as they will be first row or column that lies outside of that interval.

For example, let the robot be located in square $(2, 6)$ and the next instruction is W . Let the interval set for row 2 be $(3, 3)$ and $(5, 7)$. The robot is currently located at column 6, which is inside the interval $(5, 7)$. From this, we know that the next unvisited square in the west is $(2, 4)$, and the next unvisited square in the east is $(2, 8)$. As the instruction is W , the robot will move to square $(2, 4)$.

All that remains now is to find a way to update our interval-sets suitably whenever the robot visits a new square. Let (r, c) be the newly visited square. This can be done by updating the interval-set for both row r and column c . For both interval-sets, we will check if any existing intervals are next to the newly visited square:

- If there are two intervals adjacent to the newly visited square on both sides, we will merge these two intervals into one.
- If there is one interval adjacent to the newly visited square, we will extend this interval by 1 to include the newly visited square.
- Otherwise, we will add a new interval of length 1 containing just the newly visited square.

Since we add at most one interval in each case, the number of intervals is $O(\mathbf{N})$. Since all operations are about finding/inserting/removing a single interval, all of those can be handled easily in $O(\log(\mathbf{N}))$ time. So the overall run time of this approach is $O(\mathbf{N}\log(\mathbf{N}))$. There is also a $O(\mathbf{N})$ solution to this problem using hash tables. It is left as an exercise to the reader.